MONITOR Z · L15.7

Clase 7 — Cómo se cambia sin romper: refactorizar y probar

Cuaderno de estudio · versión del 2026-10-26

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L15 · Clase 7 — Cómo se cambia sin romper nada: refactorizar y probar

Es la última clase de teoría. Las seis anteriores explicaron **qué** cambia al partir ZTKMON. Esta explica **cómo** se cambia: de a un paso, sabiendo antes qué tiene que salir igual y con una forma de volver atrás.

> 📚 Fuentes: el JCL `ZTKMONVA` (`monitor_z/jcl/`), la salida del monitor **JOB34879** (22-10: `+HOLA`, 6 líneas `+DIAG COSTO`, 12 `+EVT`), los JCL de demo `ZTKD*`, `pc/probar_v09.py` y `plataforma_cierres/vivo/prueba_e2e.py`. *z/OS MVS Program Management: User's Guide* (SA23-1393): sentencias de control `INCLUDE`, `ENTRY` y `NAME`.

1. Refactorizar: cambiar la forma, no lo que hace

- **Refactorizar** es cambiar cómo está armado el programa **sin cambiar lo que hace**. Para el operador, el puente y el

portal, ZTKMON partido tiene que comportarse **igual** que la v0.9.

- La regla que lo hace posible: **un cambio por vez, y probado antes del siguiente**. Si se cambian tres cosas juntas y algo

falla, no se sabe cuál fue.

- **Analogía:** cambiar las cañerías de una casa **con la gente adentro**. Se cambia un tramo, se abre la canilla, se mira

que salga agua, y recién ahí se pasa al tramo siguiente.

2. Los pasos, en orden de riesgo

Sale de la L15 (punto 6). El orden va del cambio que menos puede romper al que más:

Paso	Qué se hace	Por qué en este orden
1	**Área común:** los datos pasan a la DSECT `ZTKCOM` sobre `STORAGE OBTAIN`	sigue siendo **una sola** CSECT: se aprende la técnica sin enlace entre módulos

2	`ZTKFMT` (horas, fechas, números)	el más chico que **no llama a nadie**: se aprende el enlace con algo simple
3	`ZTKMED` (métricas por job)	chico y aislado
4	`ZTKSAL` (`MANDA`, el anillo, `+HOLA`)	lo usan casi todos: conviene que la técnica ya esté probada
5	`ZTKRED` (socket y pedidos)	toca la red y el protocolo
6	`ZTKJES` (SSI 80 y la tabla de reportados)	el más grande y el que más depende de todo lo anterior



Lámina L15-26_un_paso_por_vez

3. Antes de tocar nada: la foto de la versión buena

Para decir «salió igual» hace falta saber **qué salía antes**. Esa foto ya existe: es la corrida **JOB34879** de la v0.9.

Qué mirar	Qué dio la v0.9 (JOB34879)	Qué tiene que dar la nueva
`+HOLA`	`+HOLA ZTKMON V=1 SYS=DESA JES=JES2 … ARR=JOB34879 SEQ=0000000000`	la misma forma; cambian la hora y el `ARR=`
`+EVT` de los `ZTKD*`	12 líneas: `INI` y `FIN` de 6 corridas de `ZTKDOK`, todas con `COMP=NRM CC=00`	los mismos tipos, en el mismo orden, con los mismos campos

`+DIAG COSTO` cada 60 vueltas	`CPUUS` entre 402.344 y 417.090 por cada 60 vueltas (6 líneas)	parecido; ver punto 5
`SEQ`	crece de a uno, sin saltos (último: 12)	igual

Los jobs de demo cubren los casos raros, no sólo el normal: `ZTKDOK` (RC 0), `ZTKDRC8` (RC 8), `ZTKDABN` (abend S806), `ZTKDJCL` y `ZTKDJC2` (errores de JCL), `ZTKDNUE` (un job nunca visto) y `ZTKDOKL` (el mismo nombre, más largo). En JOB34879 sólo se corrió `ZTKDOK`: **la foto de los otros seis casos todavía no existe** para la v0.9.

Analogía: la **prueba dorada**. Se guarda la salida de la versión buena y cada versión nueva se compara contra ella. Es lo mismo que hace `golden.py` con el modelo del portal (CLAUDE.md del proyecto), pero acá todavía **no hay herramienta**: la comparación de hoy sería a mano.

Antes de tocar nada: la foto de la versión buena

L15 · Clase 7 · Refactorizar y probar

```
JOB34879 · ZTKMON v0.9 · DESA · 2026-10-22
+HOLA ZTKMON V=1 SYS=DESA JES=JES2 T=2026-10-22-11:37:14 ARR=JOB34879 SEQ=0000000000
+EVT INI ZTKDOK   JOB34883 OW=PSJB CL=P T=2026-10-22-11:40:33
+EVT FIN ZTKDOK   JOB34883 OW=PSJB CL=P T=… TI=… COMP=NRM CC=00
 …   (12 +EVT: INI y FIN de 6 corridas de ZTKDOK)
+DIAG COSTO VUELTAS=00060 CPUUS=0000413414 NPOST=00007 NREP=00007 … SEQ=0000000004
 …   (6 líneas: CPUUS entre 402.344 y 417.090 cada 60 vueltas)
```

Demo	Caso	¿Hay foto de la v0.9?
ZTKDOK	job normal, RC 0	**sí · JOB34879**
ZTKDRC8	IDCAMS borra algo que no existe: RC 8	**no**
ZTKDABN	programa que no existe: abend S806	**no**
ZTKDJCL · ZTKDJC2	errores de JCL	**no**
ZTKDNUE · ZTKDOKL	job nunca visto · mismo nombre, más largo	**no**

📷 **Falta la foto de 6 casos.** Antes del paso 1 conviene correr la v0.9 con **todos** los ZTKD* y guardar su SYSPRINT. Y la comparación hoy es **a mano**: no hay una herramienta como `golden.py` para el monitor.

Fuente: SYSPRINT del monitor en JOB34879 · JCL ZTKD* (monitor_z/jcl, línea 4 de cada uno)

Lámina L15-27_la_foto

4. Qué prueba cada herramienta que ya tenés (y qué no)

Herramienta	Qué prueba	Qué **no** prueba

`pc/probar_v09.py`	conversa con el ZTKMON **real** por el socket: `SUB`, `DESDE`, `JOB`, `ACT`, `PING`, el pedido partido	es **interactiva**: no compara sola, mirás vos
`vivo/prueba_e2e.py` (13/13)	el lado **PC**: puente → portal → aprendizaje → alertas, con un corte del puente	habla con el **simulador** `sim_ztkmon.py`, **no con ZTKMON**: no prueba el Assembler
`+DIAG COSTO`	cuánta CPU gasta el monitor cada 60 vueltas	no dice si los eventos están bien
Using Map y RC del ensamblado	que cada módulo tenga lugar en su base y ensamble con RC 0	no dice nada de cómo corre

La conclusión honesta: para el Assembler, la prueba que vale es **correr el monitor nuevo con los mismos `ZTKD*` y comparar su SYSPRINT con la foto**. Hoy eso no está automatizado.

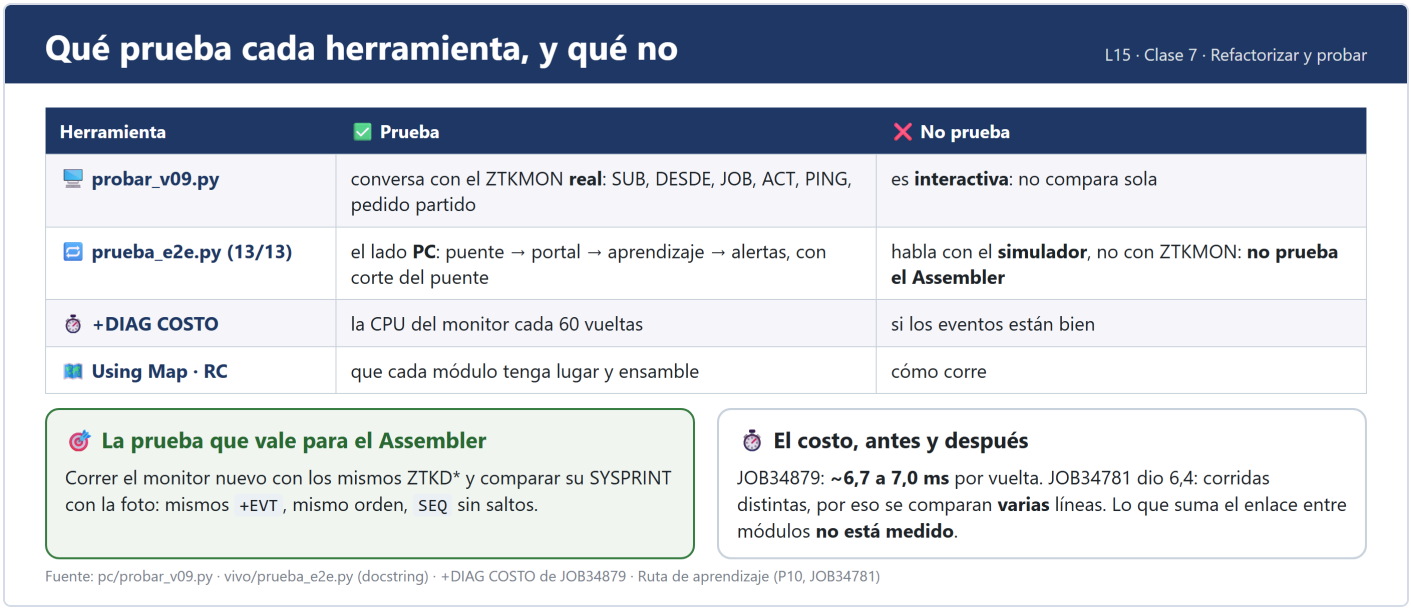


Lámina L15-28_que_prueba_cada_cosa

5. Medir el costo antes y después

- El costo propio sale del `+DIAG COSTO`. En JOB34879, con `PARM='ZTKD* 5 30 8010'` (una vuelta cada 5 s), dio entre

402.344 y 417.090 µs de CPU cada 60 vueltas, o sea unos **6,7 a 7,0 ms por vuelta**.

- **No coincide del todo** con los 6,4 ms por vuelta de JOB34781 (Ruta de aprendizaje, P10). Son corridas distintas, con

otra carga del sistema: por eso se comparan **varias** líneas y no una sola.

- **Qué esperar:** cada llamada entre módulos suma un `STM`, un `LM` y el encadenamiento (Clase 5). Frente a ~6,8 ms por

vuelta debería ser poco, pero **no está medido**. Se sabe recién con el `+DIAG COSTO` de la versión partida.

6. Siempre con vuelta atrás

- **El riesgo de hoy:** el `LKED` de `ZTKMONVA` escribe directo en `SYSLMOD DD DSN=PSJB.ZTK.LOAD(ZTKMON)`. Si se ensambla

una versión con un error, **pisa la v0.9 que funciona**.

- **La salida:** antes de cada paso, guardar la versión buena con otro nombre (por ejemplo `PSJB.ZTK.LOAD(ZTKMON9)`) o

enlazar la nueva con otro nombre y probarla con un `ZTKMONVR` que la llame. Si falla, se vuelve a la anterior sin reensamblar nada.

- **Lo mismo con el fuente:** el código está en git, en la rama `feature/ztkmon-mejoras`. Un commit por paso deja cada versión

recuperable.

Siempre con vuelta atrás, y el JCL con dos piezas

L15 · Clase 7 · Refactorizar y probar

⚠️ El riesgo de hoy

```
//SYSLMOD DD DSN=PSJB.ZTK.LOAD(ZTKMON)
```

El LKED escribe directo sobre la v0.9. Una versión con un error **pisa la que funciona**.

↩️ La salida

Guardar la buena aparte (ej. `ZTKMON9`) o enlazar la nueva con otro nombre. Y un **commit por paso** en `feature/ztkmon-mejoras`. Si falla, se vuelve sin reensamblar.

```
//* Ejemplo para entender (NO probado): dos piezas, un load module
//ASMM  EXEC PGM=ASMA90 ...   SYSIN=PSJB.ZTK.ASM(ZTKMON)  SYSLIN=&&OBJM
//ASMF  EXEC PGM=ASMA90 ...   SYSIN=PSJB.ZTK.ASM(ZTKFMT)  SYSLIN=&&OBJF
//LKED  EXEC PGM=IEWL,COND=((4,LT,ASMM),(4,LT,ASMF))
//SYSLIN DD DSN=&&OBJM,DISP=(OLD,DELETE)
//       DD DSN=&&OBJF,DISP=(OLD,DELETE)   ← los dos objetos, concatenados
//       DD *
  ENTRY ZTKMON                          ← empieza en ZTKMON
  NAME  ZTKMON(R)                       ← nombre del miembro, (R) = reemplazar
```

#	Pregunta de cada paso	Cómo se contesta
1	¿Ensambla?	RC 0 · ningún ASMA034E
2	¿Enlaza?	RC 0 · el mapa muestra todas las CSECT · ningún IEW2456E
3	¿Hace lo mismo?	los ZTKD* dan los mismos +EVT que la foto · probar_v09.py
4	¿Cuesta lo mismo?	varias líneas de +DIAG COSTO
5	¿Se puede volver?	la versión anterior aparte y el commit

Fuente: JCL ZTKMONVA (monitor_z/jcl) · z/OS MVS Program Management: User's Guide (ENTRY, NAME)

Lámina L15-29_vuelta_atras_y_jcl

7. Cómo cambia el JCL con dos piezas

Con `ZTKFMT` aparte hay **dos** fuentes y **dos** objetos. El binder tiene que recibir los dos por `SYSLIN` (Clase 4: el autocall no encuentra `ZTKFMT`, porque no está en `TCPIP.SEZATCP`). Una forma posible:

```
//ASMM     EXEC PGM=ASMA90,PARM='OBJECT,NODECK,XREF(SHORT)'
//SYSIN    DD DSN=PSJB.ZTK.ASM(ZTKMON),DISP=SHR
//SYSLIN   DD DSN=&&OBJM,DISP=(NEW,PASS),…
//ASMF     EXEC PGM=ASMA90,PARM='OBJECT,NODECK,XREF(SHORT)'
//SYSIN    DD DSN=PSJB.ZTK.ASM(ZTKFMT),DISP=SHR
//SYSLIN   DD DSN=&&OBJF,DISP=(NEW,PASS),…
//LKED     EXEC PGM=IEWL,COND=((4,LT,ASMM),(4,LT,ASMF)),PARM='LIST,MAP'
//SYSLIN   DD DSN=&&OBJM,DISP=(OLD,DELETE)
//         DD DSN=&&OBJF,DISP=(OLD,DELETE)
//         DD *
  ENTRY ZTKMON
  NAME  ZTKMON(R)
/*
```

- Cada pieza se ensambla en su paso y el binder recibe **los dos objetos concatenados**.
- `ENTRY ZTKMON`: el programa empieza en ZTKMON, no en la primera CSECT que llegue.
- `NAME ZTKMON(R)`: el nombre del miembro en `SYSLMOD`; `(R)` = reemplazar.
- El `COND` doble: si **cualquiera** de los dos ensamblados da más de 4, no se enlaza.
- **Esto es un ejemplo para entender, no un JCL probado.** Se escribe y se prueba en el paso 2. Las partes con `...` (`UNIT`, `SPACE`, `SYSLIB` de cada `ASM`) van igual que en `ZTKMONVA`.

8. La lista de cada paso

#	Pregunta	Cómo se contesta
1	¿Ensambla limpio?	RC 0 en cada `ASM`; ningún `ASMA034E`
2	¿Tiene lugar?	el *Using Map* de cada módulo
3	¿Enlaza?	RC 0 en el `LKED`; el mapa muestra todas las CSECT y ningún `IEW2456E`
4	¿Hace lo mismo?	los `ZTKD*` dan los mismos `+EVT` que la foto; `probar_v09.py` contesta `DESDE`, `JOB` y `PING`
5	¿Cuesta lo mismo?	varias líneas de `+DIAG COSTO`, comparadas con las de la foto
6	¿Se puede volver?	la versión anterior guardada aparte, y el commit en git

9. Resumen

Idea	En una línea
Refactorizar	cambiar la forma sin cambiar lo que hace
Un paso por vez	área común → `ZTKFMT` → `ZTKMED` → `ZTKSAL` → `ZTKRED` → `ZTKJES`
La foto	la salida de la v0.9 (JOB34879), contra la que se compara cada versión nueva

Las herramientas	`probar_v09.py` sí habla con ZTKMON; `prueba_e2e.py` prueba el lado PC con el simulador
El costo	`+DIAG COSTO`: ~6,7 a 7,0 ms por vuelta en JOB34879; se compara antes y después
Vuelta atrás	no pisar la versión buena en `PSJB.ZTK.LOAD`, y un commit por paso

10. Preguntas de comprobación

1. `prueba_e2e.py` da 13/13. ¿Alcanza eso para saber que el ZTKMON partido funciona? ¿Por qué sí o por qué no?
2. Hoy el `LKED` escribe en `PSJB.ZTK.LOAD(ZTKMON)`. ¿Qué riesgo tiene eso durante la refactorización, y qué harías antes del paso 1?

Con esta clase termina la teoría de la L15. Lo que sigue es el código, empezando por el paso 1, el área común.

Mis respuestas

Notas